

REFLEXIONES – PRÁCTICA UT1: RESTAURANTE PSP

EJERCICIO 2 – LANZAR UN PLATO CON `run()`

1. ¿Puede el programa principal continuar mientras `cocina.py` está trabajando?

No. Mientras `subprocess.run()` está ejecutando `cocina.py`, el programa principal queda esperando a que el proceso termine antes de continuar con las siguientes instrucciones.

2. ¿Qué comportamiento de `subprocess.run()` explica lo observado?

`subprocess.run()` es una ejecución bloqueante. Esto significa que inicia el proceso y espera a que finalice antes de devolver el control al programa principal.

3. ¿Qué programa representa el proceso padre y cuál el hijo?

`ej2.py` representa el proceso padre, ya que es el que inicia la ejecución de `cocina.py`. `cocina.py` representa el proceso hijo creado por el programa principal.

EJERCICIO 3 – CAPTURAR `stdout`, `stderr` Y `returncode`

1. ¿Qué contiene `stdout`?

`stdout` contiene la salida normal que genera el proceso, es decir, los mensajes que el programa escribe normalmente en pantalla. En este caso contiene los mensajes relacionados con la preparación de la hamburguesa.

2. ¿Qué contiene `stderr`?

`stderr` contiene los mensajes de error o información de diagnóstico que genera el proceso cuando se produce algún problema.

3. ¿Qué valor tiene `returncode` cuando la ejecución termina correctamente?

Cuando el proceso termina correctamente, `returncode` tiene el valor 0.

4. ¿Por qué puede ser útil capturar la salida en vez de mostrarla directamente?

Porque permite que el programa principal pueda analizar, guardar o mostrar posteriormente la información de una forma más controlada. También permite diferenciar la salida normal de los errores.

EJERCICIO 4 – GESTIONAR UN FALLO

1. ¿Qué cambia al utilizar `check=True`?

Con `check=True`, si el proceso termina con un código de retorno distinto de 0, `subprocess.run()` genera una excepción `subprocess.CalledProcessError`. De esta forma, el programa principal puede detectar y gestionar directamente el fallo.

2. ¿Un `returncode` distinto de 0 significa que Python se ha bloqueado?

No. Significa que el proceso ha terminado indicando algún tipo de error o situación anómala. Python no tiene por qué haberse bloqueado.

3. ¿Qué ventaja tiene gestionar el error en el programa principal?

Permite controlar el problema sin que toda la aplicación termine de forma inesperada. El programa principal puede informar del error, tomar una decisión y continuar funcionando si es posible.

EJERCICIO 5 – TIMEOUT

1. ¿Qué problema evita `timeout`?

Evita que el programa principal tenga que esperar indefinidamente a un proceso que tarda demasiado o que se ha quedado bloqueado.

2. ¿Qué ocurriría si el proceso externo quedara bloqueado y no existiera `timeout`?

El programa principal podría quedarse esperando indefinidamente a que el proceso terminara, impidiendo continuar con las siguientes operaciones.

3. ¿En qué tipo de programas sería especialmente importante utilizarlo?

Es especialmente importante en programas que dependen de procesos externos, servicios, comunicaciones o tareas que deben completarse dentro de un tiempo determinado. También resulta útil cuando una tarea podría quedarse bloqueada y no queremos que afecte indefinidamente al resto de la aplicación.

EJERCICIO 6 – COCINA CONCURRENTES CON `Popen()`

1. ¿Cuánto tardarían aproximadamente los tres platos si se ejecutaran uno detrás de otro?

Si se ejecutaran de forma secuencial, tardarían aproximadamente 15 segundos.

2. ¿Cuánto tarda aproximadamente el pedido cuando se lanzan primero los tres procesos?

Al ejecutarse los tres procesos al mismo tiempo, el tiempo total será aproximadamente el del proceso que más tarda, es decir, unos 7 segundos.

3. ¿Por qué existe esa diferencia?

Porque con `Popen()` se pueden iniciar los tres procesos sin esperar a que termine el primero. Cada preparación puede avanzar mientras las demás también están trabajando.

4. ¿Estamos observando concurrencia? Justifica la respuesta.

Sí. Los tres procesos están activos durante parte del mismo intervalo de tiempo y realizan sus tareas de forma independiente. Por eso el tiempo total se aproxima al de la preparación más lenta en lugar de a la suma de todas.

EJERCICIO 7 – CONSULTAR PROCESOS CON `poll()`

1. ¿Qué devuelve `poll()` mientras un proceso sigue activo?

Devuelve `None` mientras el proceso continúa ejecutándose.

2. ¿Qué devuelve cuando ya ha terminado?

Cuando el proceso ha terminado, devuelve su código de retorno.

3. ¿Qué diferencia existe entre `poll()` y `wait()`?

`poll()` permite comprobar el estado de un proceso sin bloquear el programa principal. `wait()`, en cambio, espera a que el proceso termine antes de continuar.

4. ¿Por qué `poll()` es apropiado para crear este monitor?

Porque permite consultar periódicamente el estado de cada preparación sin detener el programa principal. Así se puede mostrar qué platos siguen preparándose y cuáles ya han terminado.

EJERCICIO 8 – `terminate()` Y `wait()`

1. ¿Qué diferencia conceptual existe entre `terminate()` y `wait()`?

`terminate()` sirve para solicitar la finalización del proceso, mientras que `wait()` sirve para esperar a que ese proceso termine y obtener su resultado.

2. ¿Por qué utilizamos `wait()` después de `terminate()`?

Porque `terminate()` solicita que el proceso finalice, pero necesitamos asegurarnos de que realmente ha terminado antes de continuar. `wait()` permite esperar a que finalice completamente.

3. ¿Qué diferencia general existe entre `terminate()` y `kill()`?

`terminate()` solicita al proceso que termine de forma controlada, mientras que `kill()` fuerza su finalización de una manera más contundente. Por ello, `terminate()` suele ser la opción preferible cuando es suficiente para cancelar una preparación.

EJERCICIO 9 – PIPE Y `communicate()`

1. ¿Qué información viaja por `stdin`?

Por `stdin` se envía desde el proceso principal al proceso de cocina la información que necesita para realizar el pedido. En este caso, se envía el nombre del plato, por ejemplo, "Pizza carbonara".

2. ¿Qué información vuelve por `stdout`?

Por `stdout` vuelve la respuesta generada por el proceso de cocina. En este caso, indica que la preparación del plato se ha completado.

3. ¿Para qué utilizaríamos `stderr`?

`stderr` se utilizaría para enviar o recoger mensajes relacionados con errores o problemas producidos durante la ejecución del proceso.

4. ¿Por qué dos procesos independientes no pueden limitarse a compartir una variable normal llamada `pedido`?

Porque cada proceso tiene su propio espacio de memoria y una variable normal pertenece al proceso en el que se encuentra. Para intercambiar información entre procesos es necesario utilizar algún mecanismo de comunicación, como `stdin/stdout` mediante PIPE.

REFLEXIÓN FINAL – PROYECTO RESTAURANTE PSP

1. ¿Por qué has utilizado `Popen()` y no `run()` para crear los pedidos?

He utilizado `Popen()` porque el restaurante necesita crear los pedidos y volver inmediatamente al menú sin esperar a que terminen. Esto permite tener varias preparaciones funcionando al mismo tiempo y consultar posteriormente su estado.

2. ¿Qué ocurriría si ejecutaras `wait()` inmediatamente después de cada `Popen()`?

El programa esperaría a que terminara cada preparación antes de crear la siguiente. Como consecuencia, los pedidos se ejecutarían de forma prácticamente secuencial y se perdería la ventaja de trabajar con varios procesos al mismo tiempo.

3. ¿Qué función cumple `poll()` en la aplicación?

`poll()` permite consultar si cada preparación sigue activa o ya ha terminado sin bloquear el programa. De esta manera se puede mostrar al usuario el estado actual de los pedidos.

4. ¿Por qué cada preparación tiene un PID diferente?

Porque cada preparación se ejecuta como un proceso independiente. El sistema operativo asigna un identificador de proceso (PID) diferente a cada proceso para poder distinguirlos y gestionarlos por separado.

5. ¿Qué ventajas aporta ejecutar las preparaciones como procesos independientes?

Permite realizar varias preparaciones al mismo tiempo, consultar y controlar cada una de manera independiente y cancelar una preparación concreta sin tener que detener necesariamente las demás.

6. ¿Qué inconvenientes has observado?

Trabajar con varios procesos añade complejidad al programa, ya que hay que controlar sus estados, sus PID y su finalización. También hay que gestionar correctamente los procesos que puedan quedar activos para evitar que se queden funcionando sin control.

7. ¿Cómo se han comunicado los procesos cuando ha sido necesario?

La comunicación se puede realizar mediante `stdin` y `stdout` conectados con `subprocess.PIPE`, utilizando `communicate()` para enviar información al proceso hijo y recoger su respuesta.

8. Propón otro contexto real en el que este diseño pueda ser útil.

Este diseño podría utilizarse en un sistema de gestión de pedidos de una empresa de reparto. Cada pedido podría ejecutarse como un proceso independiente, permitiendo gestionar varios pedidos simultáneamente, consultar su estado y cancelar aquellos que fuera necesario.